Этот конспект сгенерирован с помощью AI. Система может допускать ошибки в формулах, вычислениях и специфической терминологии. Пожалуйста, относитесь с понимаем и проверяйте конспект!

Постановка задачи и интерпретация модели

Мы фиксируем модель как линейную: $f(x) = w^T x$, где $w \in \mathbb{R}^n$ — вектор весов, а $x \in \mathbb{R}^n$ — признаковый вектор объекта. Цель модели — предсказать знак скалярного произведения: если $w^T x > 0$, модель выдаёт +1, иначе −1. Это позволяет решать бинарную классификацию через линейную разделяющую гиперплоскость.

Функция потерь выбирается как **hinge loss** (потеря с зазором):

$$\ell(y, f(x)) = \max(0, 1 - y \cdot w^T x)$$

где $y \in -1, +1$. Эта функция штрафует ошибки и "мягко" допускает правильные предсказания вблизи границы решения.

Эмпирический риск строится как среднее по обучающей выборке из l объектов:

$$R_{\text{emp}}(w) = \frac{1}{l} \sum_{i=1}^{l} \max(0, 1 - y_i w^T x_i)$$

Задача обучения сводится к минимизации этого функционала по параметру w.

Метод решения — **градиентный спуск первого порядка**, применяемый к функции потерь. На каждом шаге обновляется вектор весов:

$$w \leftarrow w - \eta \nabla_w R_{\text{emp}}(w)$$

где $\eta > 0$ — скорость обучения (learning rate), а градиент вычисляется по формуле:

$$\nabla_w R_{\text{emp}}(w) = -\frac{1}{l} \sum_{i=1}^{l} y_i x_i \cdot \mathbf{1}_{y_i w^T x_i < 1}$$

То есть градиент ненулевой только для тех объектов, которые **ошибочно классифицированы** или находятся вблизи границы.

Алгоритм:

```
for epoch in range(num_epochs):
    for i in range(l):
        if y[i] * (w.T @ x[i]) < 1:
            w = w - eta * y[i] * x[i]
```

Важно: градиентный спуск **не гарантирует сходимость** к глобальному минимуму, особенно при шумных данных или плохой инициализации. Поэтому применяются эвристики — например, **стохастический градиентный спуск (SGD)**, где на каждом шаге используется один случайный объект из выборки вместо всей суммы.

Метрика: качество модели оценивается на **отдельной тестовой выборке** размера k. Средняя ошибка (например, доля неправильных предсказаний) сравнивается с ошибкой на обучении.
- Если разница мала — переобучение отсутствует.
- Если ошибка на тесте в разы выше — модель переобучилась, требуется регуляризация или упрощение архитектуры.

Архитектура: линейная модель $f(x) = w^T x$, пороговая классификация по знаку результата.
Метрика: accuracy (доля правильных предсказаний) на тестовой выборке.

Мы решаем задачу бинарной классификации с помощью линейной модели $f(x) = w^T x$, где предсказание определяется знаком результата: если $f(x) > 0$ — класс +1, иначе -1. Обучение проводится на обучающей выборке, а проверка качества — на тестовой (отложенной), состоящей из k объектов с известными метками. Полная выборка содержит $l + k$ объектов. После обучения вычисляются средние ошибки: на обучении и на контроле.
Сравнение этих метрик позволяет оценить степень переобучения: если ошибка на тесте лишь немного выше — переобучение слабое; если в разы или на порядок больше — модель переобучилась, требуется регуляризация или упрощение архитектуры.
Метрика: accuracy (доля правильных предсказаний) на тестовой выборке.
Архитектура: линейная модель $f(x) = w^T x$ с пороговой классификацией по знаку результата.

Модель и функция потерь

Это значит, что мы в пространстве PH строим разделяющую гиперплоскость. Эта гиперплоскость имеет направляющий вектор — это самый W. Если объект попал в ту половину пространства, куда смотрит вектор W, то он относится к классу плюс 1; можно сказать, что угол между вектором x и вектором w острый. В противном случае, если скалярное произведение отрицательно (угол тупой), объект относится к классу минус один. Таким образом, линейные модели для классификации определяют разделяющую гиперплоскость в пространстве признаков.

Следующий шаг — договориться о функции потерь. Для задачи классификации самое простое определение ошибки: она происходит, когда правильный ответ y и предсказание модели a имеют разные знаки. Поскольку оба значения — либо +1, либо -1, проверка знака эквивалентна вычислению произведения $a \cdot y$. При этом $a = \mathrm{sign}(x^\top w)$, то есть функция sign применяется к скалярному произведению.

Можно не использовать явную функцию sign: величина $x^\top w \cdot y$ сама по себе хорошо характеризует ошибку — если она отрицательна, ошибка есть; если положительна или равна нулю — ошибки нет. Однако проблема в том, что при построении функционала потерь (суммы по всем объектам выборки) получаем дискретную величину: либо 0, либо 1 на объект. Сумма принимает значения от 0 до $l + 1$, то есть становится **дискретным** функционалом. Минимизация такого функционала — задача комбинаторной оптимизации, которая является NP-трудной и неудобна для решения.

Поэтому мы идём на «жульничество»: заменяем бинарную функцию потерь на **непрерывную аппроксимацию сверху** $\mathcal{L}(x^\top w \cdot y)$. Требование к функции $\mathcal{L}$ — она должна быть непрерывной, а лучше гладкой и убывающей: чем больше значение $x^\top w \cdot y$, тем меньше потеря; чем оно ближе к нулю или отрицательному — тем хуже. Такая замена позволяет перейти от дискретной оптимизации к непрерывной, используя градиентные методы.

Теперь мы минимизируем гладкую верхнюю оценку исходного функционала потерь, что даёт приближённое решение задачи минимизации числа ошибок. Важно помнить, что в машинном обучении всегда есть тестовая выборка, на которой проверяется предсказательная сила модели.

Данный подход — аппроксимация бинарной функции потерь непрерывной — является стандартным приёмом при работе с линейными моделями классификации.

Организуем функционал q — этот функционал будет очень неприятным для решения оптимизационной задачи, поскольку он принимает $l+1$ дискретное значение. Действительно, сумма по всем объектам выборки от величины, которая бинарна (либо 0, либо 1), даёт значения: 0, 1, 2 и так далее — вплоть до l. Значит, это дискретный функционал, а задача минимизации дискретного функционала — это задача комбинаторной оптимизации, как правило NP-трудная. Решать её не очень приятно.

Поэтому мы идём на «жульничество»: подменяем бинарную функцию потерь на непрерывную оценку сверху. Обозначим эту новую функцию как L. Она должна быть непрерывной, а лучше — гладкой и убывающей. Условие к L: чем больше значение агрегата $x^T wy$, тем меньше потеря; если оно отрицательно — это плохо, и тогда потеря растёт. Таким образом, мы заменяем исходную пороговую функцию потерь на её гладкую верхнюю оценку.

Теперь вместо дискретной задачи минимизации числа ошибок мы решаем непрерывную задачу: минимизируем $L(x^T wy)$. Это приближённая минимизация исходного функционала — число ошибок модели. Важно помнить, что в машинном обучении всегда есть тестовая выборка, на которой проверяется предсказательная сила модели.

Этот приём — аппроксимация бинарной функции потерь непрерывной — является стандартным при работе с линейными моделями классификации. Он позволяет использовать градиентные методы оптимизации, такие как стохастический градиентный спуск или Adam, вместо перебора всех возможных решений.

Идея замены дискретной функции потерь на гладкую обобщается на любые модели: не только линейные, но и нелинейные, основанные на принципе дискриминации. В общем случае модель строит разделяющую поверхность — гиперплоскость (если $g(x) = x^T w$) или более сложную форму (если $g(x)$ — параметрическая функция из произвольного семейства). Объекты классифицируются по одну или другую сторону этой поверхности.

Ключевое понятие — **margin** (отступ): это расстояние от точки до разделяющей поверхности, взятое со знаком. Если объект правильно классифицирован — отступ положительный; если ошибочно — отрицательный. Геометрически: $\mathrm{margin}(x) = y(x^T w)$, где $y \in -1, +1$.

Практически важно анализировать распределение отступов на обучающей выборке: - Большой положительный margin — объект типичен для своего класса, лежит глубоко внутри него. - Большой отрицательный margin — объект ошибочно классифицирован,

возможно, аномалия или ошибка измерения. - Малый (близкий к нулю) margin — пограничный случай, требует внимания.

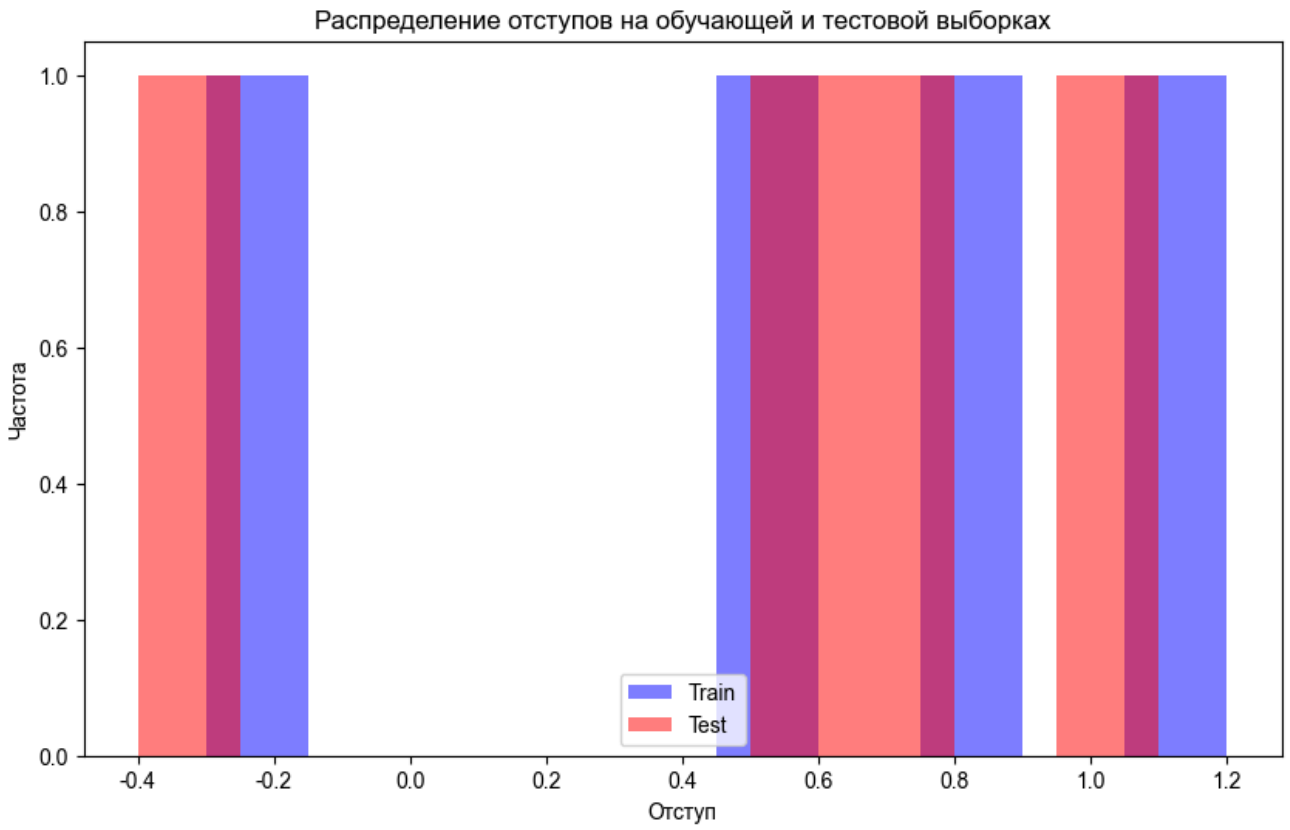
Распределение отступов позволяет оценить надёжность классификации: если кривая отступов пересекает ноль под острым углом — разделение устойчивое; если лежит близко к нулю — модель неустойчива, высокий риск переобучения. Сравнение распределения отступов на обучающей и тестовой выборках помогает выявить масштаб переобучения.

Функции потерь могут быть разными: hinge loss, log-loss (cross-entropy), squared loss и др. Несмотря на разнообразие подходов, все они сводятся к минимизации эмпирического риска:

$$\min_w \frac{1}{n} \sum_{i=1}^{n} L(y_i f(x_i; w))$$

где $f(x; w)$ — предсказание модели, L — функция потерь. Выбор конкретной формы L зависит от задачи и свойств данных, но основная цель — обеспечить гладкость и возможность применения градиентных методов.

Метрика: margin используется как количественная оценка качества разделения классов и устойчивости классификации.
Алгоритм: минимизация гладкой верхней оценки бинарной функции потерь (например, hinge loss) реализуется через `model.fit()` с оптимизатором Adam или SGD



Распределение отступов на обучающей и тестовой выборках

.

Архитектура: линейная модель $f(x; w) = x^T w$ — частный случай дискриминантного классификатора, но идея применима к любым параметрическим моделям.

Линейная модель $f(x; w) = x^T w + b$ была предложена Маккалоком и Питтсом в 1943 году как упрощённая математическая интерпретация работы нервной клетки: входные сигналы (признаки) суммируются с весами, соответствующими синаптическим связям, и при достижении порога генерируется выход. Эта модель отражает аддитивность электрического заряда — каждый дендрит вносит вклад пропорционально своему признаку и весу w_i, а аксон передаёт результат следующему нейрону.

Несмотря на упрощённость, линейная модель стала основой для построения искусственных нейронных сетей: последовательное соединение таких элементов позволяет моделировать сложные нелинейные зависимости. Современные глубокие сети содержат сотни миллионов нейронов — в разы больше, чем в мозге кошки (~1 млрд), но пока не достигают масштаба человеческого мозга (86 млрд). Тем не менее, вычислительные возможности растут экспоненциально: за последние 30 лет количество обучаемых параметров удваивается примерно каждые два года, что

позволяет масштабировать модели до сотен миллиардов параметров на высокопараллелизованных серверах.

На практике линейная модель остаётся важным базовым случаем для анализа: её структура $f(x; w) = x^T w + b$ легко интерпретируется и используется как элемент более сложных архитектур. При этом выбор функции потерь (Loss) определяет поведение модели — например, кросс-энтропия подходит для вероятностной классификации, hinge loss — для маржинальных методов, а параболическая функция (как в линейном дискриминанте Фишера) штрафует объекты за нахождение внутри класса, что может быть нежелательно.

Таким образом, разнообразие методов классификации сводится к вариациям функции потерь при фиксированной структуре модели: $\min_w \mathcal{L}(y, f(x; w))$. Это позволяет гибко настраивать поведение классификатора под конкретную задачу, сохраняя общую парадигму минимизации эмпирического риска.

Обучение: градиентные методы и оптимизация

Итак, мы дошли до того, что... А как же обучать? Вот функционал мы уже можем выписать. Значит, мы через `L_i` коротко будем обозначать функцию потерь на item-объекте — это функция от вектора параметров `w`, и даже не важно, как она устроена: элита как-то выбрана функция потерь, и даже не очень важно, что там за модель — один нейрон или может быть целая сеть. И в общем то о чём я сейчас прямо буду говорить — про градиентный метод оптимизации.

Когда мы так записали, мы уже абстрагировались от того, как устроена наша модель и как устроена функция потерь. Давайте протянем эту абстракцию дальше: пусть `L_i(w)` — это функция, которая зависит от параметров нашей модели `w` каким-то образом. Главное, чтобы она была непрерывно дифференцируемой. Пока что мы от неё больше ничего требовать не будем.

Очень важно, что сумма по всем объектам обучающей выборки строится из отдельных функций потерь для каждого объекта:

$$\mathcal{L}(w) = \sum_{i=1}^{N} L_i(w),$$

где `x_i` и `t_i` — вектор признаков и метка соответственно для i-го объекта. Поэтому функция `L_i(w)` как функция от вектора параметров `w` индивидуальна для каждого объекта `i`.

Теперь вспомним, что такое градиентные методы оптимизации: это очень просто. В каждой точке мы можем посчитать вектор частных производных — он называется градиентом:

$$\nabla_w \mathcal{L}(w),$$

и он показывает направление наибольшего возрастания функции в данной точке. А нам нужно минимизировать, значит, идти не по градиенту, а в противоположном направлении — антиградиенте.

Начинаем с какого-то начального приближения `w_0`, и каждое следующее приближение вычисляем как:

$$w_{k+1} = w_k - \eta \nabla_w \mathcal{L}(w_k),$$

где `η` — величина шага, пока просто некоторый параметр метода.

Что мы делаем? Мы должны посчитать сумму по всем объектам обучающей выборки и сместиться на такое суммарное значение. И тут самая интересная идея: не нужно суммировать по всему миллиону объектов. По закону больших чисел можно взять лишь часть слагаемых — например, 10 или даже 100 случайных объектов из миллиона.

Есть экстремистская реализация этой идеи: а давайте возьмём только одно слагаемое? То есть вместо суммы по всем объектам будем использовать градиент только по одному случайному объекту:

$$w_{k+1} = w_k - \eta \nabla_w L_i(w_k),$$

где `i` выбирается случайно из всей обучающей выборки.

После обновления параметров `w` мы снова выбираем новый случайный объект и вычисляем градиент уже по нему. Такой подход приводит к тому, что смещения параметров напоминают броуновское движение: нас немножечко шатает, но в целом мы бежим в сторону убывания функционала потерь — причём ускоренного.

Эта идея оказалась очень продуктивной, несмотря на то, что выглядит простым «читерством». Она приводит к ускорению сходимости и широко используется на практике. Сейчас рассмотрим самый простой вариант: случайный выбор одного

объекта из обучающей выборки и обновление параметров только по его градиенту — это и есть стохастический градиентный спуск (`SGD`).

В методе стохастического градиентного спуска (`SGD`) на каждом шаге выбирается один случайный объект из обучающей выборки, вычисляется его градиент функции потерь и обновляются параметры модели — вектор весов `W`. Вместо полного суммирования градиентов по всей выборке (что требует $O(N)$ операций на итерацию), используется лишь одно слагаемое, что обеспечивает значительное ускорение вычислений. При этом благодаря случайному порядку выбора объектов процесс избегает зацикливания и движется в направлении антиградиента, хотя траектория обновления напоминает броуновское движение из-за случайных скачков.

Алгоритм `SGD` реализуется следующим образом:
- Инициализируются веса (например, нулевым вектором или малым случайным шумом).
- В цикле по итерациям:
- Случайно выбирается один объект из обучающей выборки.
- Вычисляется градиент функции потерь только по этому объекту.
- Параметры обновляются по формуле: $W_{t+1} = W_t - \eta \nabla L(W_t; x_i, y_i)$, где η — темп обучения (learning rate).
- Для отслеживания сходимости используется **экспоненциальное скользящее среднее** значений функции потерь вместо точного усреднения по всей выборке. Это позволяет сохранить эффективность метода и избежать повторной полной оценки Loss на каждом шаге.

Формула экспоненциального скользящего среднего:

$$q_t = \lambda q_{t-1} + (1 - \lambda) L(W_t)$$

где q_t — текущее взвешенное среднее, $\lambda \in [0, 1]$ — параметр забывания (decay rate). При $\lambda \to 1$ последнее значение получает больший вес; при $\lambda = 0.95$, например, вклад данных за 20 шагов назад уменьшается в $0.95^{20} \approx 0.36$.

Такой подход позволяет эффективно оценивать динамику Loss без потери преимуществ стохастичности и ускоряет процесс оптимизации на больших выборках.

Если мы выбрали параметр λ, то это эквивалентно усреднению по последним $1/\lambda$ итерациям. Например, при $\lambda = 0.05$ — усреднение примерно по 20 последним итерациям, при $\lambda = 0.01$ — по 100. Параметр λ называется темпом забывания

предыстории ряда и часто используется для сокращения вычислений при необходимости усреднения последних значений — будь то арифметическое или взвешенное. Такой приём применяется не только в оптимизации, но и в прогнозировании временных рядов (что будет рассмотрено позже).

Можно ли избежать случайного «шарахания» градиента по одному объекту и двигаться более целенаправленно? Да — достаточно усреднять градиенты по последним итерациям. Один из популярных способов — использование экспоненциального скользящего среднего, что соответствует методу моментум (momentum), предложенному Борисом Теодоровичем Поликом в конце 1960-х и до сих пор активно используемому. Этот подход будет подробно рассмотрен при изучении нейронных сетей.

Метод Stochastic Average Gradient (SAG) основан на явном усреднении градиентов по всем объектам обучающей выборки, но без их полного суммирования. Вместо этого для каждого объекта хранится вектор градиента; при повторном выборе объекта из выборки его старый вклад вычитается из накопленной суммы, а новый — добавляется. Таким образом, сумма градиентов обновляется эффективно: $$\mathbf{g}*i^{(t)} = \nabla*. ЯвноесуммированиеLвекторовразмерностиNExtra close brace or missing open brace, иобновлениепроисходиткак\mathbf{G} := \mathbf{G} - \mathbf{g}_i^{(t-1)} + \mathbf{g}_i^{(t)}

Главное отличие SAG от SGD: градиентный шаг делается по сумме градиентов всех объектов, но вычисление этой суммы осуществляется инкрементально. Всё остальное в алгоритме остаётся таким же — необходимо уметь вычислять градиенты функции потерь $\mathcal{L}(\mathbf{x}, y; \mathbf{w})$, которая непрерывно дифференцируема по параметрам $\mathbf{w}$.

Метрика: качество обучения оценивается через значение Loss на обучающей и валидационной выборках.
Архитектура: используется линейная модель с явным хранением градиентов по объектам.
Алгоритм: Stochastic Average Gradient (SAG) — усреднение градиентов по всем объектам без полного суммирования.

Теперь рассмотрим недостатки подхода и пути их устранения. Первая «подковерная» тема — инициализация весов $\mathbf{w}$. Рекомендация «занулить все веса» не всегда хороша: в старых работах по нейронным сетям часто предлагается инициализировать веса малыми случайными значениями. Причина — при больших значениях весов и использовании функций потерь с горизонтальными асимптотами (например, сигмоид)

может возникать эффект паралича сети: градиенты близки к нулю на асимптотах, что приводит к обнулению шагов обновления. Чтобы этого избежать, веса следует инициализировать не слишком далеко от нуля.

Существует простая эвристика для выбора масштаба весов: если задача — регрессия с квадратичной функцией потерь и признаки стахастически независимы, можно получить аналитическое приближение коэффициентов $\mathbf{w}$. Оно выглядит так:

$$\hat{\mathbf{w}}_j = \frac{\langle \mathbf{y}, \mathbf{x}_j \rangle}{|\mathbf{x}_j|_2^2}$$

где $\mathbf{y}$ — вектор целевых значений размерности L, $\mathbf{x}_j$ — столбец признаков для признака j. Это решение работает в пространстве размерности RL, и при выполнении предположения о независимости признаков даёт неплохое приближение. Однако это предположение часто не выполняется на практике, поэтому эвристика может давать неточные результаты.

В контексте выбора начального приближения для весов в градиентных методах, одной из ключевых эвристик является использование аналитического решения задачи наименьших квадратов при условии независимости признаков. При решении регрессии с квадратичной функцией потерь и предположении о стационарности признаков, начальное значение веса w_j можно вычислить как:

$$w_j = \frac{\langle \mathbf{y}, \mathbf{x}_j \rangle}{|\mathbf{x}_j|_2^2}$$

где $\mathbf{y}$ — вектор целевых значений размерности L, $\mathbf{x}_j$ — столбец признаков для признака j. Это решение работает в пространстве размерности RL и даёт неплохое приближение при выполнении предположения о независимости признаков. Однако на практике это предположение часто не выполняется, поэтому эвристика может давать неточные результаты.

Для улучшения качества обучения предлагается использовать мультистарт: запускать процесс оптимизации с несколькими случайными начальными приближениями параллельно и выбирать лучшее или несколько лучших решений для дальнейшего усреднения или продолжения обучения. Это особенно эффективно при возможности масштабирования вычислений.

При выборе объектов для обновления весов в стохастическом градиентном спуске (SGD) рекомендуется не использовать полностью случайный выбор, а применять стратегию, ориентированную на пограничные случаи. Вместо случайных объектов следует выбирать те, у которых отступ (margin) минимален — то есть объекты, находящиеся вблизи границы решения или ошибочные. Это позволяет фокусироваться на трудных примерах и ускорять сходимость.

Альтернативная стратегия — отбирать не самые плохие, а почти лучшие объекты (например, 9 из 10 случайных), исключая выбросы с сильно отрицательным отступом. Такие аномалии могут мешать обучению и их исключение способствует более стабильной оптимизации в зоне пограничного решения.

Выбор размера шага градиентного спуска также требует эвристического подхода: теоретические рекомендации (например, $\eta = 1/T$) на практике часто неэффективны. Более практичный способ — использовать скорейший градиентный спуск, решая одномерную оптимизационную задачу по шагу численно, например, методом половинного деления с 3–5 итерациями. Это позволяет адаптировать размер шага под текущую геометрию функции потерь без жёстких теоретических ограничений.

Выбираем функцию потерь, убывающую и имеющую горизонтальную правую ветвь — это означает, что объекты, хорошо классифицируемые (лежащие в толще своего класса), дают очень малое значение градиента. Однако уточнять классификатор на таких объектах нецелесообразно: мы теряем эффективность обучения. Вместо этого следует выбирать примеры, находящиеся в пограничной зоне или ошибочные — то есть те, где модель ещё не научилась корректно работать.

Для отбора таких объектов можно использовать эвристику: из 10 случайных выборок (или меньшего числа — например, 3–5) выбирается тот, у которого отступ минимален. Альтернативно — отбрасывать объекты с сильно положительным отступом или, наоборот, не брать те, у которых отступ сильно отрицательный. Это позволяет исключить выбросы и аномалии из обучающей выборки: на них модель всё равно не улучшится, но они мешают обобщению.

Ключевой интерес представляет настройка в пограничной зоне, соответствующая желтому участку на графике распределения отступов.

Что касается выбора градиентного шага: теоретические рекомендации (например, $\eta = 1/T$) на практике часто неэффективны. Более практичный подход — использовать скорейший градиентный спуск, решая одномерную оптимизационную задачу по шагу численно, например методом половинного деления с 3–5 итерациями. Это позволяет

адаптировать размер шага под текущую геометрию функции потерь без жёстких теоретических ограничений.

Ещё одна идея — приближённо использовать метод второго порядка. Например, метод Ньютона-Рафсона требует вычисления и обращения матрицы Гессиана — вычислительно дорого. Однако если предположить диагональное преобладание в матрице вторых производных (что часто выполняется), можно обнулить смешанные производные и оставить только диагональные элементы. Обратная к такой матрице — просто обратная величина на диагонали. Эта приближённая техника работает хорошо вблизи минимума, где вторая производная ускоряет движение в сторону антиградиента.

Для устойчивости при $H \approx 0$ (в окрестности точки перегиба) добавляют небольшое положительное число μ:

$$\eta = \frac{1}{H + \mu}$$

Это позволяет плавно переключаться между поведением первого и второго порядка — вдали от минимума движение остаётся быстрым, как у градиентного спуска, а вблизи оптимума — ускоряется, как у метода Ньютона.

Архитектура: линейная модель $f(x) = \mathrm{sign}(w^\top x)$.
Алгоритм: стохастический градиентный спуск с адаптивным шагом через одномерную оптимизацию или приближённый метод второго порядка.
Метрика: отступ (margin), доля правильно классифицированных объектов, значение функции потерь.

Переобучение в линейных моделях возникает из-за мультиколлинеарности признаков — ситуации, когда среди признаков есть линейно зависимые или почти линейно зависимые. Это означает существование ненулевых весов α_i, таких что взвешенная сумма соответствующих признаков для любого объекта равна нулю:

$$\sum_{i=1}^{d} \alpha_i x_i = 0$$

Например, при кредитном скоринге признаки «доход заемщика», «доход жены» и «доход семьи» могут быть линейно зависимы (третий — сумма первых двух). Это

приводит к неединственности решения w: существует бесконечно много векторов весов, дающих одинаковую классификацию.

Причина переобучения — избыточная свобода модели при наличии таких зависимостей: она может «подогнать» веса под обучающую выборку, но плохо обобщать на новые данные.

В окрестности точки перегиба функционал действительно приближается к линейному, и в этом случае градиентный метод первого порядка остаётся эффективным. Параметр μ позволяет адаптировать поведение метода: вблизи минимума он обеспечивает медленное движение, близкое к методу второго порядка, а на больших расстояниях — быстрое движение, характерное для обычного градиентного спуска. Эта техника является приближённой и вычислительно лёгкой: вторая производная не требуется явно, поскольку достаточно работать с диагональной матрицей Гессе, обратная которой вычисляется просто как обратная величина на диагонали.

Ключевая проблема машинного обучения — переобучение. В линейных моделях оно возникает из-за мультиколлинеарности признаков: когда среди признаков существуют линейно- или почти линейно-зависимые. Например, признак «доход семьи» может быть суммой признаков «доход мужа» и «доход жены». Это приводит к неединственности решения w: существует бесконечно много векторов весов, дающих одинаковую классификацию.

Причина переобучения — избыточная свобода модели при наличии таких зависимостей: она может «подогнать» веса под обучающую выборку, но плохо обобщать на новые данные. Если для объекта x^T взвешенная сумма признаков с коэффициентами даёт 0 (т.е. $\sum w_i x_i = 0$), то любое масштабирование $w \to w + \gamma u$, где $u^T x = 0$, не меняет значение функции классификации:

$$a(x) = \mathrm{sign}(w^T x) = \mathrm{sign}((w + \gamma u)^T x) = \mathrm{sign}(w^T x),$$

поскольку $u^T x = 0$. При этом γ может быть сколь угодно большим, что приводит к нестабильности весов и ухудшению качества на новых данных.

Такая ситуация возможна по двум причинам:
1. Число объектов в выборке меньше числа признаков — тогда среди них обязательно найдутся линейно-зависимые;
2. Признаки явно зависимы (например, один — сумма других).

Это порождает линейное многообразие решений $w + \gamma u$, и задача переобучения сводится к выбору «лучшего» решения из этого семейства.

Для борьбы с этим эффектом предлагается модифицировать функционал потерь: добавить регуляризатор, минимизирующий норму весов. Конкретно — ввести штраф за величину $|w|^2$. Тогда новая функция потерь: $$ L(w) = \underbrace{L_{\text{data}}(w)} *{\text{ошибка на данных}}* + *\underbrace{\lambda* |w|^2}, $$ где $\lambda > 0$ — параметр регуляризации.}

Градиент по w теперь включает дополнительный член:

$$\nabla_w L(w) = \nabla_w L_{\text{data}}(w) + 2\lambda w.$$

Таким образом, градиентный спуск автоматически «тянет» веса к нулю, выбирая из бесконечного семейства решений то, у которого норма минимальна — это и является стандартной формой **L2-регуляризации (Ridge)**.

Если количество объектов в выборке меньше числа признаков, то среди них обязательно существует линейная зависимость — это классический источник переобучения. Подобная ситуация приводит к тому, что градиентный спуск не может однозначно выбрать решение из бесконечного семейства эквивалентных по функции потерь $L_{\text{data}}(w)$, поскольку множество решений образует линейное многообразие: $w + \gamma v$, где $\gamma \in \mathbb{R}$.

Чтобы решить эту проблему, вводится регуляризация — дополнительный член в целевой функции, который ограничивает норму весов. Конкретно, к исходной функции потерь добавляется квадратичный штраф по вектору весов:

$$L(w) = L_{\text{data}}(w) + \lambda |w|^2.$$

Градиент такой модифицированной функции становится:

$$\nabla_w L(w) = \nabla_w L_{\text{data}}(w) + 2\lambda w,$$

что приводит к обновлению весов вида:

$$w_{t+1} = w_t - \eta\,(\nabla_w L_{\text{data}}(w_t) + 2\lambda w_t).$$

Это эквивалентно шагу, в котором текущий вектор w_t домножается на коэффициент $(1 - 2\eta\lambda) < 1$, то есть происходит **weight decay** — постепенное «затухание» весов к нулю.

Параметр регуляризации λ (в транскрипте обозначенный как τ) контролирует силу штрафа: слишком большое значение приводит к тривиальному решению $w = 0$, слишком малое — не даёт эффекта контроля переобучения и мультиколлинеарности. Его выбор осуществляется через **holdout** или **кросс-валидацию**, поскольку оптимальный масштаб заранее неизвестен.

Метод стохастического градиентного спуска (SGD) остаётся простым в реализации: достаточно уметь вычислять градиенты по данным, а при обработке одного объекта — обновлять веса и «забывать» его. Это позволяет работать с потоковыми данными без хранения всей выборки, что было особенно важно в эпоху ограниченных вычислительных ресурсов.

Однако такой подход требует тонкой настройки параметров и применения эвристик: SGD сам по себе не гарантирует стабильность или оптимальность, но остаётся мощным инструментом благодаря своей эффективности и масштабируемости.

Следующий этап — переход к вероятностной интерпретации моделей классификации, включая формулировку функций потерь через логарифмическую вероятность и связь с методами оптимизации на основе градиентов.

Оценка результата: переобучение, регуляризация и риск

Предполагается, что наша выборка — это выборка пар X и Y, порожденная случайно и независимо из некоторого распределения в пространстве этих пар. Совместную плотность $p(x, y)$ можно представить как условную вероятность $p(y|x)$, умноженную на маргинальную плотность $p(x)$. Параметрическая модель строится только для условной вероятности: $p(y|x; w)$, где w — вектор параметров модели. Это означает, что мы не моделируем распределение X, а сосредоточены исключительно на зависимости Y от X. Такой подход называется **дискриминативным**, поскольку цель — построить разделяющую поверхность между классами.

Принцип максимума правдоподобия применяется к условной вероятности: выбирается значение параметров w, при котором наблюдаемая выборка становится наиболее вероятной. Функция правдоподобия для независимых объектов выборки — произведение плотностей $p(y|x; w)$ по всем объектам. Поскольку логарифм — монотонная функция, оптимизацию ведут по логарифму правдоподобия:

$$\mathcal{L}(w) = \sum_{i=1}^{n} \log p(y_i | x_i; w)$$

При этом $p(x_i)$ не зависит от w, поэтому при оптимизации его можно исключить — он не влияет на выбор параметров.

Для задачи бинарной классификации ($y \in -1, +1$) условная вероятность может быть записана через сигмоиду:

$$p(y = +1 | x; w) = \sigma(w^T x), \quad p(y = -1 | x; w) = 1 - \sigma(w^T x)$$

Тогда логарифм правдоподобия принимает вид:

$$\mathcal{L}(w) = \sum_{i=1}^{n} \left[y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i)) \right]$$

что эквивалентно минимизации **кросс-энтропийной потери**. Эта запись — частный случай обобщённой формулировки через условную вероятность.

Ключевой момент: оба подхода — вероятностный (максимизация правдоподобия) и дискриминативный (минимизация эмпирического риска с функцией потерь L) — приводят к одной и той же оптимизационной задаче, записанной как сумма по объектам выборки. Это позволяет переходить между интерпретациями:
- из вероятностной модели получить функцию потерь,
- или из функции потерь — построить вероятностную модель.

Таким образом, **максимизация правдоподобия** и **минимизация аппроксимированного эмпирического риска** — это разные формулировки одной задачи обучения. Оба подхода могут решаться с помощью `sgd`, `adam` или других методов градиентного спуска, поскольку целевая функция является суммой по обучающей выборке.

Теперь, когда мы учли, что вектор параметров $\mathbf{w}$ сам является случайной величиной, порождаемой из некоторого вероятностного распределения с гиперпараметром γ, совместное правдоподобие записывается как:

$$\log P(\mathcal{X}, \mathbf{w}) = \log P(\mathcal{X} | \mathbf{w}) + \log P(\mathbf{w} | \gamma)$$

где $\mathcal{X}$ — обучающая выборка, $P(\mathcal{X}|\mathbf{w})$ — условная вероятность данных при фиксированном $\mathbf{w}$ (которая соответствует функции правдоподобия в стандартной постановке), а $P(\mathbf{w}|\gamma)$ — априорное распределение на параметры модели.

Метрика:
Максимизация совместного правдоподобия эквивалентна решению задачи:

$$\max_{\gamma,\mathbf{w}} \left[\sum_{i=1}^{n} \log P(y_i|\mathbf{x}_i, \mathbf{w}) + \log P(\mathbf{w}|\gamma) \right]$$

Это — двухуровневая вероятностная модель: на верхнем уровне выбирается распределение параметров $\mathbf{w}$, на нижнем — данные генерируются из модели с этим $\mathbf{w}$.

Алгоритм:
При этом регуляризация в виде $\lambda|\mathbf{w}|^2$ (weight decay) имеет прямую вероятностную интерпретацию: она соответствует выбору гауссовского априора на параметры,

$$P(\mathbf{w}|\gamma) = \mathcal{N}(\mathbf{w}; 0, \sigma^2 I), \quad \text{где } \lambda = \frac{1}{2\sigma^2}$$

Таким образом, L_2-регуляризация — это следствие предположения о нормальности параметров.

Архитектура:
В этом подходе модель становится иерархической: данные порождаются через параметры модели, а параметры — через гиперпараметры. Это позволяет формализовать **регуляризацию как часть вероятностной структуры**, а не просто как штраф в функции потерь.

Таким образом, переход от минимизации эмпирического риска к вероятностной интерпретации регуляризации показывает, что **любая регуляризация может быть обоснована априорным распределением на параметры модели** — и наоборот, любое априорное распределение порождает соответствующую функцию потерь.

В рамках вероятностной интерпретации обучения модели вводится двухуровневая порождающая модель: сначала случайным образом выбирается вектор параметров $\mathbf{W}$ из некоторого распределения с гиперпараметром γ, затем при фиксированном $\mathbf{W}$

генерируется выборка данных $(x_i, y_i)_{i=1}^n$. Совместное правдоподобие данных и модели записывается как $P(\mathcal{D}, \mathbf{W}) = P(\mathcal{D} \mid \mathbf{W}) \cdot P(\mathbf{W})$, где $P(\mathcal{D} \mid \mathbf{W})$ — условная плотность данных при заданном параметре модели (например, функция правдоподобия линейной регрессии), а $P(\mathbf{W})$ — априорное распределение на параметры.

Применение принципа максимума апостериорной вероятности (MAP) к совместному распределению приводит к оптимизации логарифма совместного правдоподобия:

$$\log P(\mathcal{D}, \mathbf{W}) = \log P(\mathcal{D} \mid \mathbf{W}) + \log P(\mathbf{W})$$

Первый слагаемый соответствует эмпирическому риску (например, кросс-энтропии в классификации), второй — регуляризатору. Таким образом, **регуляризация на вероятностном языке интерпретируется как априорное распределение на параметры модели**.

Если $P(\mathbf{W})$ — многомерное гауссовское распределение с нулевым математическим ожиданием и одинаковой дисперсией по всем координатам:

$$P(\mathbf{W} \mid c) = \frac{1}{(2\pi c)^{d/2}} \exp\left(-\frac{|\mathbf{W}|^2}{2c}\right),$$

то после логарифмирования и приведения к виду функционала потерь получаем:

$$\log P(\mathbf{W}) = -\frac{1}{2c}|\mathbf{W}|^2 + \text{const},$$

что эквивалентно L^2-регуляризации (weight decay) с коэффициентом $\lambda = 1/(2c)$. Это демонстрирует, что **введение гауссовского априора на параметры модели формально порождает L^2-штраф в функции потерь**.

Аналогично, если $P(\mathbf{W})$ — распределение Лапласа:

$$P(\mathbf{W} \mid b) = \frac{1}{(2b)^d} \exp\left(-\frac{|\mathbf{W}|_1}{b}\right),$$

то после логарифмирования получаем:

$$\log P(\mathbf{W}) = -\frac{|\mathbf{W}|_1}{b} + \text{const},$$

что соответствует L^1-регуляризации (Lasso). Таким образом, **выбор между L^2 и L^1 регуляризацией определяется формой априорного распределения на параметры**: гауссовское — для L^2, лапласовское — для L^1.

Метрика:
В случае бинарной классификации логистическая регрессия использует функцию потерь — логарифмическую (log loss):

$$\mathcal{L}(\mathbf{W}) = -\sum_{i=1}^{n} \left[y_i \log \sigma(\mathbf{w}_i^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}_i^\top \mathbf{x}_i)) \right],$$

где $\sigma(z) = \frac{1}{1+e^{-z}}$ — сигмоидная функция. Эта потеря является гладкой аппроксимацией сверху от hinge loss и обеспечивает дифференцируемость при оптимизации методом градиентного спуска (`model.fit()`).

Архитектура:
Логистическая регрессия строится как линейная модель с сигмоидной активацией на выходе: $\hat{y} = \sigma(\mathbf{W}^\top \mathbf{x})$. Это базовый бейзлайн для задач классификации, который рекомендуется применять в первую очередь при появлении новой задачи.

Алгоритм:
Оптимизация проводится методом стохастического градиентного спуска (`sgd`) или его адаптивных модификаций (`adam`), где на каждом шаге обновляются параметры по градиенту функции потерь с учётом регуляризатора:

```
optimizer.step()
```

в рамках цикла `model.fit()`.

Вектор параметров $\mathbf{W}$ имеет гауссовское распределение с центром в нуле и одинаковой дисперсией по всем координатам. Это предположение можно ослабить: например, разрешить разные дисперсии или сделать ковариационную матрицу многомерного нормального распределения недиагональной — тогда между компонентами параметров появятся взаимозависимости. Однако если предположить отсутствие зависимостей (матрица диагональна), математическое ожидание нулевое и

все компоненты независимы, то такой априор эквивалентен L_2-регуляризации: штраф за сумму квадратов весов пропорционален энтропии гауссовского распределения.

Если же вместо суммы квадратов регуляризуем сумму модулей параметров, это соответствует предположению о том, что вектор $\mathbf{W}$ имеет распределение Лапласа. Распределение Лапласа также колоколообразное, но с более острыми хвостами — в отличие от гауссовского, где хвосты экспоненциально спадают, у Лапласа они степенные. Это приводит к тому, что малые значения весов поощряются сильнее, чем при L_2, а большие — наказываются жёстче. Такой выбор регуляризатора соответствует L_1-норме и часто используется для получения разреженных моделей.

Метрика: Эффективность выбора между L_1 и L_2 регуляризацией будет оцениваться в последующих лекциях по сравнению их влияния на обобщающую способность модели, устойчивость к шуму и интерпретируемость признаков. Пока же акцент делается на понимании вероятностной интерпретации этих штрафов.

Логистическая регрессия — это частный случай линейной дискриминантной модели, применяемый как базовый бейзлайн для задач классификации. Она строится на двухклассовой постановке: $y \in -1, +1$, и использует линейный отступ $s = \mathbf{w}^\top \mathbf{x} + b$. Функция потерь — логарифмическая (логарифмическая потеря), гладкая аппроксимация пороговой функции ошибки. Она непрерывно дифференцируема и позволяет применять градиентные методы оптимизации.

Функция потерь для логистической регрессии имеет вид:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - \log(1 + e^{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}) \right]$$

Ключевой результат: любая линейная дискриминантная модель с заданной функцией потерь может быть переинтерпретирована как вероятностная модель. В случае логистической регрессии вероятность принадлежности объекта к классу $y = +1$ выражается через сигмоиду:

$$P(y = +1|\mathbf{x}) = \sigma(s) = \frac{1}{1 + e^{-s}}, \quad s = \mathbf{w}^\top \mathbf{x} + b$$

Отступ s интерпретируется как мера уверенности: при $s \to +\infty$, $P(y = +1|\mathbf{x}) \to 1$; при $s \to -\infty$ — стремится к 0. На разделяющей поверхности ($s = 0$) вероятности равны 0.5. Таким образом, функция потерь определяет не только способ оптимизации, но и закон преобразования отступа в вероятность.

Регуляризованная логистическая регрессия записывается как:

$$\min_{\mathbf{w},b} \left[-\frac{1}{N} \sum_{i=1}^{N} \log P(y_i|\mathbf{x}_i) + \lambda R(\mathbf{w}) \right]$$

где $R(\mathbf{w}) = |\mathbf{w}|_2^2$ — L_2-регуляризация (weight decay), а $\lambda > 0$ — коэффициент регуляризации.

Архитектура: Многоклассовая логистическая регрессия обобщает двухклассовую модель на произвольное число классов. Для каждого класса $k = 1, \ldots, K$ вводится свой направляющий вектор $ \mathbf{w}*k . Предсказаниеосуществляетсяпоправилу :$ \hat{y} = \arg\max) $$} (\mathbf{w}_k^\top \mathbf{x

Вероятности классов пересчитываются с помощью функции softmax: $$ P(y=k | \mathbf{x}) = \frac{\exp(\mathbf{w}*k^\top \mathbf{x})}{\sum $$}^K \exp(\mathbf{w}_j^\top \mathbf{x})

Softmax преобразует вектор маржинальных значений (скалярных произведений) в нормированный вероятностный вектор, что позволяет трактовать модель как параметрическую модель условной вероятности $P(y|\mathbf{x})$. Это делает метод совместимым с принципом максимума правдоподобия.

Алгоритм: Оптимизация многоклассовой логистической регрессии проводится аналогично двухклассовому случаю — методом стохастического градиентного спуска (`sgd`) или `adam`, где на каждом шаге обновляются параметры по градиенту функции потерь с учётом регуляризатора:

```
optimizer.step()
```

в рамках цикла `model.fit()`.

Метрика: Логистическая регрессия широко применяется в задачах кредитного скоринга и других областях, где важна интерпретируемость модели: каждый коэффициент w_j показывает вклад признака x_j в логарифм шансов (odds ratio), что позволяет анализировать влияние факторов на принятие решения.

Вероятность принадлежности объекта к классу — вещественная величина, принимающая значения на отрезке [0, 1], и является непрерывной функцией. Это свойство объединяет логистическую регрессию с методами регрессии: для каждого класса можно предсказать вероятность его принадлежности, что позволяет обобщить

модель на произвольное число классов. В многоклассовой постановке для каждого класса вводится собственный направляющий вектор w_j, по которому вычисляется скалярное произведение с входным вектором признаков. К классу относится тот, у которого значение маржина (margin) максимально.

Для преобразования вектора маржинов в вероятностное распределение используется функция **Softmax**: $$ \text{Softmax}(z)$i = \frac{\exp(z_i)}{\sum, \quad i = 1, \dots, K, $$ где z_i — скалярное произведение для класса i, K — число классов. Результат — нормированный вектор вероятностей, интерпретируемый как распределение $P(Y|X)$.}^K \exp(z_j)

Постановка задачи многоклассовой логистической регрессии с регуляризацией аналогична двумерному случаю: оптимизация проводится по принципу максимума правдоподобия с добавлением регуляризатора (например, L2), где для каждого класса свой вектор весов w_j размерности, равной числу признаков. Регуляризация применяется ко всем векторам одновременно.

Метрика: Логистическая регрессия широко применяется в задачах кредитного скоринга и других областях, где важна интерпретируемость модели: каждый коэффициент w_j показывает вклад признака x_j в логарифм шансов (odds ratio), что позволяет анализировать влияние факторов на принятие решения.

В задачах оценки рисков модель используется для построения **скоринговой карты** — линейной модели над бинаризованными признаками. Даже если исходные признаки вещественные или категориальные, они преобразуются в бинарные (например, возраст разбивается на диапазоны). На основе вероятностного вывода строится оценка математического ожидания потерь $\mathbb{E}[D_{XY}]$, где D_{XY} — прибыль/убыток при исходе Y для объекта X. При положительном исходе (кредит возвращён) $D_{XY} > 0$, при отрицательном (не возвращён или частично) — $D_{XY} < 0$.

Для обеспечения устойчивости портфеля применяется метод **Value at Risk (VaR)**: с помощью вероятностной модели для каждого заемщика из портфеля (включающего миллионы объектов) проводится стохастическое моделирование исходов. Например, разыгрывается 10 000 сценариев потерь на всех заемщиках, строится эмпирическое распределение итоговых убытков и проверяется, укладывается ли правый хвост распределения (например, 1% самых худших случаев) в положительную область. Это гарантирует, что даже при крайне неблагоприятном сценарии банк не выйдет в минус.

Архитектура: Модель — линейная по признакам с функцией Softmax на выходе; обучение проводится через стохастический градиентный спуск (`sgd`) или его

модификации (`adam`, `momentum`) в рамках `model.fit()` с регуляризацией (weight decay).
Алгоритм: Оптимизация функции потерь кросс-энтропии с L2-регуляризацией по всем векторам весов w_j.
Метрика: Оценка риска через математическое ожидание и Value at Risk на основе вероятностного вывода.